



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Desarrollo de un Monitor Básico del sistema en C

Proyecto Final

Presenta:

Rojo Peña Manuel Ianluck

Profesor:

Javier León Cotonieto

Ayudante laboratorio:

Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:

27 de mayo, 2026

Objetivo

El objetivo de este proyecto es implementar un **monitor de sistema** en lenguaje C enfocado en el monitoreo del procesador y la memoria. La herramienta debe exponer al menos dos métricas dinámicas que se actualicen en tiempo real, y puede presentarse como una aplicación de línea de comandos o con interfaz gráfica básica.

Este reporte documenta el desarrollo incremental de dicha herramienta, organizado por **implementaciones**, cada implementación representa una etapa funcional completa que puede extenderse a lo largo de más de una sesión de trabajo. El criterio que tomamos de avance no es el tiempo, sino la funcionalidad que se agrega en cada implementación.

El plan de implementaciones es el siguiente:

1. **Primer Implementación. Escaneo e impresión básica:** lectura de `/proc/meminfo` y `/proc/[pid]/` para obtener métricas de memoria y lista de procesos, salida por `printf`. Se establece también la infraestructura de compilación con CMake y el entorno reproducible con Docker.
2. **Segunda Implementación. Terminal interactiva con ncurses:** sustitución del `printf` por una interfaz de terminal estilizada que se actualiza en tiempo real. Se introducen dos hilos POSIX, uno recolector de datos y uno de dibujado, de modo que la interfaz nunca se congele independientemente de la carga del sistema.
3. **Tercer Implementación. Interfaz gráfica con GTK-4:** migración de la presentación a una ventana de escritorio con tablas, barras de progreso y gráficas de historial, manteniendo el mismo backend de recolección en C.
4. **Cuarta Implementación. Monitoreo de red (extensión futura):** incorporación de captura y análisis de paquetes de red al estilo de `iftop`, mostrando emisor, receptor, protocolo y volumen de tráfico.

Primer Implementación: Escaneo e impresión básica

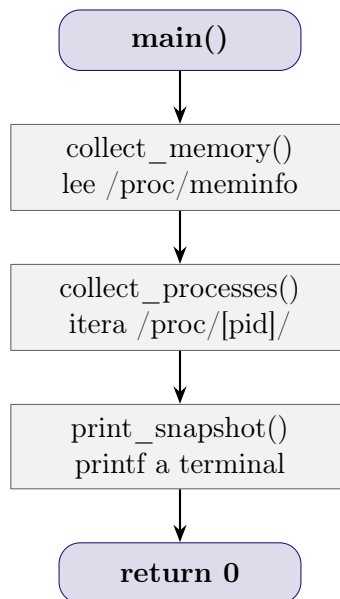
El objetivo de esta primera implementación fue establecer la base del proyecto, definir las estructuras de datos que representan el estado del sistema, leer las métricas de memoria y la lista de procesos activos desde el sistema operativo, e imprimirlos en la terminal con `printf`. No hay interfaz especial, no hay actualizaciones en tiempo real y no hay hilos. Se trata de un *escaneo único*, el programa arranca, toma una fotografía del sistema y termina. La razón: antes de introducir complejidad concurrente (hilos, mutex) o de presentación (ncurses, GTK), es necesario verificar que los datos que se leen son correctos. Con un `printf` simple nos permitimos confirmar que las métricas tienen sentido antes de construir encima de ellas.

Arquitectura del código

El proyecto se organiza en tres archivos fuente y un header:

- `include/monitor.h`: definición de las estructuras de datos y declaración de las funciones públicas.
- `src/monitor.c`: implementación de la recolección de datos y la impresión.
- `src/main.c`: punto de entrada; orquesta las tres llamadas en secuencia.

El flujo completo del programa es lineal:



Estructuras de datos: `monitor.h`

La primera decisión de diseño fue centralizar todos los datos del sistema en dos estructuras: `ProcessInfo` y `SystemSnapshot`.

`ProcessInfo` representa un único proceso, sus campos mapean directamente con los archivos que el kernel expone en `/proc/[pid]/`:

```

1 //Numero maximo de procesos que el snapshot puede almacenar
2 #define MAX_PROCS 512
3
4 /* Representa un unico proceso del sistema operativo,
5    cada campo viene de una fuente distinta dentro de /proc/[pid]/ */
6 typedef struct
7 {
8     int process_id;           //PID: identificador unico del proceso, fuente
9     int parent_process_id;    //PPID: PID del proceso que lo creo, fuente
10    char name[256];           //Nombre corto del ejecutable, max 15 chars
11    char cmdline[512];        //Comando completo con argumentos
12    char parent_name[256];     //Nombre del proceso padre
13    char state;               //Estado actual: R running, S sleeping, D waiting
14                               //I/O, Z zombie, T stopped
15    long vm_size_kb;          //Tamano total del espacio virtual en KB, incluye
16                               //todo lo reservado aunque no esteen RAM
17    long memory_kb;           //RSS en KB: paginas realmente en RAM ahora, es
18                               //la metrica de uso real de memoria
19 }
20
21 ProcessInfo;

```

Código 1: monitor.h struct ProcessInfo (líneas 12-21)

Cada campo tiene una fuente específica dentro de /proc:

- process_id viene del nombre del directorio en /proc;
- name de /proc/[pid]/comm; command_line de
- /proc/[pid]/cmdline; state, parent_process_id,
- memory_kb y vm_size_kb de /proc/[pid]/status.

SystemSnapshot agrupa las métricas globales de memoria y el arreglo de procesos en una sola estructura que se pasa por referencia entre funciones:

```

1 /* Fotografia completa del estado del sistema en un instante
2    agrupa metricas de memoria global y la lista de procesos */
3 typedef struct
4 {
5     long memory_total_kb;     //RAM total instalada
6     long memory_used_kb;      //RAM en uso real = total - free - buffers -
7                               //cached
8     long memory_free_kb;      //RAM completamente libre
9     long memory_buffers_kb;   //Cache de metadatos de filesystem
10    long memory_cached_kb;     //Cache de contenido de archivos
11    long swap_total_kb;        //Espacio de intercambio total
12    long swap_used_kb;         //Swap en uso = SwapTotal - SwapFree
13    ProcessInfo processes[MAX_PROCS]; //Arreglo estatico de procesos
14    encontrados
15    int process_count;         //Cuantas entradas validas hay en
16    processes[]
17 }

```

```
15 SystemSnapshot;
```

Código 2: monitor.h struct SystemSnapshot (líneas 23–34)

El arreglo `processes[MAX_PROCS]` es estático con `MAX_PROCS = 512`, de este modo evitamos asignación dinámica de memoria (`malloc`) en esta primera etapa, simplificando el código a costa de un límite fijo. En un sistema de escritorio típico el número de procesos rara vez supera los 400.

Restricción a Linux y portabilidad entre distribuciones

La herramienta depende de `/proc`, un sistema de archivos virtual exclusivo del kernel Linux, lo que significa que no compila ni corre en macOS, Windows ni BSD sin modificaciones.

Sin embargo, `/proc` está presente en todas las distribuciones basadas en Linux (Fedora, Ubuntu, Debian, Arch, Alpine) porque es una interfaz del kernel, no del sistema de paquetes, el mismo código que lee `/proc/meminfo` en Fedora funciona sin cambios en Ubuntu o Debian. El estándar que define su estructura está documentado en `man 5 proc` y ha sido estable durante décadas.

Para garantizar reproducibilidad entre máquinas con distintas distribuciones se introdujo un `Dockerfile`, descrito al final de este documento.

Lectura de memoria `collect_memory()`

La función `collect_memory()` lee el archivo `/proc/meminfo`, que el kernel regenera en cada acceso con los valores actuales de memoria. Su formato es una lista de campos con el patrón `Campo: valor kB`:

```
1  /* Funcion para leer /proc/meminfo y extrae las metricas de memoria del
   * sistema /proc/meminfo
   * es generado por el kernel en cada lectura; cada linea tiene formato "
   * Campo: valor kB" */
2
3  void
4  collect_memory(SystemSnapshot *system_snapshot)
5  {
6      FILE *f = fopen("/proc/meminfo", "r"); //abrimos el archivo virtual del
   * kernel
7      if (!f)
8          return; //Si no se puede abrir, salir sin modificar el snapshot
9
10     char line[128]; //buffer para leer una linea a la vez
11     long value; //variable temporal donde sscanf deposita el numero
12
13     // Leemos linea por linea hasta EOF
14     while (fgets(line, sizeof(line), f)) {
15         /* sscanf intenta hacer match del patron en la linea
16          * Devuelve el numero de campos que pudo leer (0 o 1)
17          * solo guardamos el valor si el match fue exitoso (== 1) */
18         if (sscanf(line, "MemTotal: %ld kB", &value) == 1)
19             system_snapshot->memory_total_kb = value;
```

```

20     else if (sscanf(line, "MemFree: %ld kB", &value) == 1)
21         system_snapshot->memory_free_kb = value;
22     else if (sscanf(line, "Buffers: %ld kB", &value) == 1)
23         system_snapshot->memory_buffers_kb = value;
24     else if (sscanf(line, "Cached: %ld kB", &value) == 1)
25         system_snapshot->memory_cached_kb = value;
26     else if (sscanf(line, "SwapTotal: %ld kB", &value) == 1)
27         system_snapshot->swap_total_kb = value;
28     else if (sscanf(line, "SwapFree: %ld kB", &value) == 1)
29         system_snapshot->swap_used_kb = system_snapshot->swap_total_kb -
30         value;
31 }
32 fclose(f); //siempre cerrar el archivo para liberar el descriptor
33
34 /* Calculamos memoria realmente usada buffers y cached son reutilizables
35    por el kernel cuando hay presion de memoria, por eso no se cuentan
36    como "en uso real" por aplicaciones */
37 system_snapshot->memory_used_kb = system_snapshot->memory_total_kb
38                                 - system_snapshot->memory_free_kb
39                                 - system_snapshot->memory_buffers_kb
40                                 - system_snapshot->memory_cached_kb;
41 }

```

Código 3: monitor.c — collect_memory() (líneas 18–47)

El archivo `/proc/meminfo` tiene un formato fijo garantizado por el kernel, la cadena `"MemTotal: %ld kB"` es el patrón literal que aparece en el archivo, incluyendo los espacios y la unidad kB. `sscanf` devuelve el número de campos que pudo leer exitosamente; al verificar `== 1` nos aseguramos de que el match fue completo antes de guardar el valor, si la línea no coincide con el patrón, `sscanf` devuelve 0 y simplemente pasamos a la siguiente. Se cumple de manera análoga para los demás casos.

Linux usa la RAM libre para cachear contenido de archivos (`Cached`) y metadatos de filesystem (`Buffers`), esta memoria aparece como “usada” pero el kernel la libera inmediatamente cuando una aplicación la necesita. La fórmula correcta es:

$$\text{used} = \text{total} - \text{free} - \text{buffers} - \text{cached}$$

Esta es la misma fórmula que usan `htop` y `free -h`. Es por esta razón de qué `memory_used` no es simplemente `total - free`.

Lectura de procesos collect_processes()

```

1 /* Funcion para iterar /proc buscando directorios numericos (cada uno es
   un PID)
2    y extrae informacion de cada proceso vivo */
3 void
4 collect_processes(SystemSnapshot *system_snapshot)
5 {

```

```

6 DIR *d = opendir("/proc"); //abrimos el directorio /proc como un
  iterador
7 if (!d)
8     return;
9
10 struct dirent *entry; //entrada actual del directorio
11 system_snapshot->process_count = 0;
12
13 /* readdir devuelve la siguiente entrada cada vez que se llama,
14    NULL cuando no quedan mas entradas */
15 while ((entry = readdir(d)) != NULL && system_snapshot->process_count <
16     MAX_PROCS) {
17
18     /* Solo nos interesan entradas cuyo nombre empieza con digito
19        /proc tiene directorios como "net", "sys", "self" que ignoramos */
20     if (!isdigit(entry->d_name[0]))
21         continue;
22
23     int pid = atoi(entry->d_name);
24     ProcessInfo p;
25     memset(&p, 0, sizeof(p)); //inicializamos todos los campos a cero
26     p.process_id = pid;
27
28     char path[64]; //buffer para construir rutas como /proc/1234/comm
29     char line[256]; //buffer para leer lineas de los archivos de status
30     FILE *f;
31
32     /* Contiene el comando completo con argumentos separados por '\0'
33        ej: "/usr/bin/firefox\0--no-sandbox\0"
34        fread en lugar de fgets porque puede haber '\0' en medio */
35     snprintf(path, sizeof(path), "/proc/%d/cmdline", pid);
36     f = fopen(path, "r");
37     if (f) {
38         int len = fread(p.cmdline, 1, sizeof(p.cmdline) - 1, f);
39         fclose(f);
40         //Reemplazamos separadores '\0' por espacios para poder imprimirlo
41         for (int i = 0; i < len - 1; i++)
42             if (p.cmdline[i] == '\0') p.cmdline[i] = ' ';
43         p.cmdline[len] = '\0';
44     }
45
46     /* Nombre corto del ejecutable, truncado a 15 chars por el kernel
47        siempre disponible, incluso para hilos del kernel */
48     snprintf(path, sizeof(path), "/proc/%d/comm", pid);
49     f = fopen(path, "r");
50     if (!f)
51         continue; //si no podemos leer comm, el proceso ya termino saltar
52     fgets(p.name, sizeof(p.name), f);
53     p.name[strcspn(p.name, "\n")] = '\0'; //quitamos el newline final
54     fclose(f);
55
56     /* Archivo con multiples campos en formato "Campo:\tvalor"
57        leemos State, PPid, VmRSS y VmSize de aqui */
58     snprintf(path, sizeof(path), "/proc/%d/status", pid);

```

```

58     f = fopen(path, "r");
59     if (!f)
60         continue;
61     while (fgets(line, sizeof(line), f)) {
62         long val;
63         char c;
64         int iv;
65         if (sscanf(line, "State:\t%c", &c) == 1)
66             p.state = c;
67         else if (sscanf(line, "PPid:\t%d", &iv) == 1)
68             p.parent_process_id = iv;
69         else if (sscanf(line, "VmRSS:\t%ld kB", &val) == 1)
70             p.memory_kb = val;
71         else if (sscanf(line, "VmSize:\t%ld kB", &val) == 1)
72             p.vm_size_kb = val;
73     }
74     fclose(f);
75
76     /* Filtramos procesos del kernel: no tienen memoria de usuario (RSS =
77     0)
78     hilos como kworker, migration, rcu_sched no aportan informacion
79     util */
80     if (p.memory_kb == 0)
81         continue;
82
83     //Leemos el comm del proceso padre usando su PPID
84     if (p.parent_process_id > 0) {
85         snprintf(path, sizeof(path), "/proc/%d/comm", p.parent_process_id);
86         f = fopen(path, "r");
87         if (f) {
88             fgets(p.parent_name, sizeof(p.parent_name), f);
89             p.parent_name[strcspn(p.parent_name, "\n")] = '\0';
90             fclose(f);
91         }
92     }
93
94     //Guardamos el proceso en el arreglo y avanzamos el contador
95     system_snapshot->processes[system_snapshot->process_count] = p;
96     system_snapshot->process_count++;
97 }
98
99     closedir(d); //Liberamos el descriptor del directorio
100 }

```

Código 4: monitor.c collect_processes() (líneas 49-122)

En Linux cada proceso vivo tiene un directorio en `/proc` cuyo nombre es su PID, por ejemplo `/proc/1234/`. No existe una llamada de sistema que devuelva directamente la lista de PIDs activos; la forma estándar (la misma que usa `ps` y `top`) es iterar `/proc` y filtrar las entradas cuyo nombre sea numérico con `isdigit()`.

A diferencia de los demás archivos de `/proc`, `cmdline` no usa saltos de línea como separador entre argumentos usa el carácter nulo `'\0'`. Si usáramos `fgets`, la lectura se detendría en

el primer '\0' y perderíamos todos los argumentos, fread lee los bytes crudos y luego reemplazamos los '\0' internos por espacios para poder imprimirlos.

Los hilos internos del kernel (kworker, migration, rcu_sched) aparecen como entradas en /proc pero no tienen espacio de memoria de usuario asignado. Su VmRSS es 0, incluirlos no aporta información útil para un monitor de memoria y solo ocupan entradas del arreglo.

VmRSS (Resident Set Size) son las páginas del proceso que están físicamente en RAM en este momento. VmSize es todo el espacio de direcciones virtuales reservado, que puede incluir regiones que el kernel nunca ha cargado a RAM o que ya fueron movidas a swap. Para saber quién está *realmente* usando memoria, RSS es la métrica correcta.

Impresión y ordenamiento print_snapshot()

```
1  /* Funcion para imprimir el snapshot completo, primero metricas de memoria
2     luego tabla de procesos ordenada de mayor a menor RSS */
3  void
4  print_snapshot(const SystemSnapshot *system_snapshot)
5  {
6     /* --- seccion de memoria --- */
7     printf("\n===== Memoria =====\n");
8     printf("  Total:    %10ld KB  (%ld MB)\n",
9         system_snapshot->memory_total_kb, system_snapshot->memory_total_kb /
10        1024);
11     printf("  Usada:    %10ld KB  (%ld MB)\n",
12         system_snapshot->memory_used_kb, system_snapshot->memory_used_kb /
13        1024);
14     printf("  Libre:    %10ld KB  (%ld MB)\n",
15         system_snapshot->memory_free_kb, system_snapshot->memory_free_kb /
16        1024);
17     printf("  Buffers: %10ld KB  (%ld MB)\n",
18         system_snapshot->memory_buffers_kb, system_snapshot->
19         memory_buffers_kb / 1024);
20     printf("  Cache:    %10ld KB  (%ld MB)\n",
21         system_snapshot->memory_cached_kb, system_snapshot->memory_cached_kb
22         / 1024);
23     printf("  Swap:    %10ld KB usados / %ld KB total\n",
24         system_snapshot->swap_used_kb, system_snapshot->swap_total_kb);
25
26     /* --- procesos ordenados por RSS --- */
27     //Usamos memcpy para no modificar el arreglo original del snapshot;
28     //qsort ordena en sitio usando cmp_mem como criterio
29     ProcessInfo sorted[MAX_PROCS];
30     memcpy(sorted, system_snapshot->processes, system_snapshot->
31         process_count * sizeof(ProcessInfo));
32     qsort(sorted, system_snapshot->process_count, sizeof(ProcessInfo),
33         cmp_mem);
34
35     printf("\n===== Procesos con memoria (%d)
36         =====\n",
37         system_snapshot->process_count);
```

```

30 printf("%-6s  %-6s  %-16s  %-10s  %-10s  %-8s  %-15s  %s\n",
31        "PID", "PPID", "Nombre", "RSS KB", "VmSize KB", "Estado", "Padre", "
    Comando");
32 printf("%-6s  %-6s  %-16s  %-10s  %-10s  %-8s  %-15s  %s\n",
33        "-----", "-----", "-----", "-----",
34        "-----", "-----", "-----", "
    -----");
35
36 for (int i = 0; i < system_snapshot->process_count; i++) {
37     const ProcessInfo *p = &sorted[i];
38
39     //Truncamos cmdline a 40 chars, si esta vacio usar name como fallback
40     char cmd_trunc[41];
41     snprintf(cmd_trunc, sizeof(cmd_trunc), "%.40s",
42             p->cmdline[0] ? p->cmdline : p->name);
43
44     printf("%-6d  %-6d  %-16s  %-10ld  %-10ld  %-8s  %-15s  %s\n",
45           p->process_id,
46           p->parent_process_id,
47           p->name,
48           p->memory_kb,
49           p->vm_size_kb,
50           state_str(p->state),
51           p->parent_name,
52           cmd_trunc);
53 }
54
55 //Resumen final
56 if (system_snapshot->process_count > 0) {
57     printf("\n Mayor consumidor: %s (PID %d) %ld KB\n",
58           sorted[0].name, sorted[0].process_id, sorted[0].memory_kb);
59     printf(" Menor consumidor: %s (PID %d) %ld KB\n",
60           sorted[system_snapshot->process_count-1].name,
61           sorted[system_snapshot->process_count-1].process_id,
62           sorted[system_snapshot->process_count-1].memory_kb);
63 }
64 }

```

Código 5: monitor.c print_snapshot() (líneas 124–180)

Antes de imprimir, la función copia el arreglo de procesos con `memcpy` y lo ordena con `qsort` usando `cmp_mem` como comparador, se ordena una *copia* para no alterar el orden original del snapshot, lo cual será importante cuando en implementaciones futuras otros hilos lean esos datos concurrentemente.

El comparador `cmp_mem` usa el idioma $(b > a) - (b < a)$ en lugar de una resta directa $b - a$. La razón es que `memory_kb` es de tipo `long` y la resta podría producir *overflow* si los valores son muy grandes, corrompiendo el ordenamiento silenciosamente.

Punto de entrada: main.c

```

1 #include "monitor.h"

```

```

2 #include <string.h>
3
4 /* Punto de entrada del programa */
5 int
6 main()
7 {
8     SystemSnapshot system_snapshot;
9     /* Inicializamos toda la estructura a cero antes de usarla, sin esto los
10        campos
11        tendrían basura de memoria valores aleatorios que podrían hacer
12        fallar los cálculos */
13     memset(&system_snapshot, 0, sizeof(system_snapshot));
14
15     //Paso 1: leemos métricas de memoria desde /proc/meminfo
16     collect_memory(&system_snapshot);
17     //Paso 2: iteramos /proc y llenar el arreglo de procesos
18     collect_processes(&system_snapshot);
19     //Paso 3: imprimimos todo lo recolectado
20     print_snapshot(&system_snapshot);
21
22     return 0;
23 }

```

Código 6: main.c completo (líneas 1–17)

main() inicializa el snapshot con `memset` a cero antes de cualquier lectura, sin esto, los campos del struct contendrían basura de la pila y los cálculos de memoria producirían valores incorrectos. La secuencia es estrictamente lineal: recolectar memoria, recolectar procesos, imprimir.

Infraestructura de compilación: CMake

El proyecto usa CMake con dos archivos `CMakeLists.txt`, uno raíz que define el proyecto y estándar de C, y uno dentro de `src/` que lista los archivos fuente y enlaza las bibliotecas. Esta separación nos permitirá agregar nuevos módulos en futuras implementaciones modificando solo el `CMakeLists.txt` de `src/`.

La compilación completa (sin uso de contenedores) se realiza con:

```
cmake -B build && cmake --build build && ./build/src/sysmon
```

Entorno reproducible: Dockerfile

Un reto práctico de cualquier proyecto en C es la gestión de dependencias del sistema, en esta implementación las dependencias son mínimas (solo GCC y CMake), pero en implementaciones futuras se agregarán **ncurses** para la terminal interactiva y **GTK-4 con libadwaita** para la interfaz gráfica. Instalar estas bibliotecas varía entre distribuciones, el paquete se llama `libgtk-4-dev` en Debian/Ubuntu pero `gtk4-devel` en Fedora por ejemplo.

Para eliminar esta fricción se introdujo un `Dockerfile` basado en `debian:bookworm-slim`, se eligió Debian porque es la base más neutral y ampliamente compatible, y sobre Fedora

porque los paquetes de GTK-4 están igualmente actualizados en los repositorios de Debian. La imagen instala desde ahora las bibliotecas que se usarán en implementaciones futuras (`libncurses-dev`, `libgtk-4-dev`, `libadwaita-1-dev`) , aunque en esta etapa el binario no las enlaza. Esto documenta las dependencias del proyecto completo y evita modificar el Dockerfile en cada implementación.

```
1 FROM debian:bookworm-slim
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 RUN apt-get update && apt-get install -y \
6     gcc \
7     cmake \
8     libncurses-dev \
9     libgtk-4-dev \
10    libadwaita-1-dev \
11    pkg-config \
12    && rm -rf /var/lib/apt/lists/*
13
14 WORKDIR /app
15 COPY . .
16
17 RUN cmake -B build && cmake --build build
18
19 CMD ["/build/src/sysmon"]
```

Código 7: Dockerfile

Para ejecutar con acceso a los procesos reales del sistema huésped:

```
docker build -t sysmon .
docker run --rm --pid=host sysmon
```

La opción `-pid=host` comparte el espacio de nombres de procesos del sistema huésped con el contenedor. Sin ella, el contenedor tiene su propio `/proc` aislado y solo vería sus propios procesos internos.